Namespace ASE_Assignment

Classes

Draws

This class extends the ICanvas Interface, it contains methods that allow initalization and usage of a bitmap to enable users to draw shapes.

Form1

Main class of the program. It contains the graphical elements of the interface that the user sees alongside methods that initialize and handle user input

ownArray

Class that implements array functionality

ownCommandfactory

The class extends CommandFactory from BOOSE. Both classes receive commands from the user as input and create new objects of their corresponding classes, if they exist in the factory.

ownCompoundCommand

class used to find the types of command

ownElse

class responsible for elses in the program when if conditions return false

ownEnd

class which is reponsible for ending if, while and for blocks of code

ownFor

class which allows for loops in the application

ownIf

class that enables if conditions in the program

ownInt

Class used to handle integer variables.

ownParser

ownPeek

class used to access elements from an array and return their values

ownPenColour

Class used to create a pen of a specified colour

ownPoke
: class used to set values at specific index in an array

ownReal

ownSqrt
: class that allows users to square root a number

ownStoredProgram

ownWhile

ownWrite
: class to allow writing text on the screen

Class Draws

Namespace: ASE_Assignment

Assembly: ASE Assignment.dll

This class extends the ICanvas Interface, it contains methods that allow initalization and usage of a bitmap to enable users to draw shapes.

```
public class Draws : ICanvas
```

Inheritance

object ← Draws

Implements

ICanvas

Inherited Members

object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.MemberwiseClone() , object.ReferenceEquals(object, object) , object.ToString()

Remarks

The class implements pens, brushes, bitmaps and graphics that other parts of the program rely on. It is the class that handles what processed user input is drawn on.

Properties

PenColour

penColour variable setter and getter

```
public object PenColour { get; set; }
```

Property Value

object

Xpos

xPos variable setter and getter

```
public int Xpos { get; set; }
```

Property Value

int

Ypos

yPos variable setter and getter

```
public int Ypos { get; set; }
```

Property Value

int

getInstance

```
public static Draws getInstance { get; }
```

Property Value

Draws

Methods

Circle(int, bool)

A method that draws a circle at the current X and Y position with a given radius.

```
public void Circle(int radius, bool filled)
```

Parameters

`radius` int

Integer which specifies the radius of the circle from the current X and Y position.

`filled` bool

Boolean which decides whether the circle should be filled inside or hollow.

Exceptions

CanvasException

CanvasException is thrown when the radius is negative.

Clear()

Removes any element from the canvas and sets the screen to white.

```
public void Clear()
```

DrawTo(int, int)

A method that draws a line from the current X and Y position to a different X and Y position specified by the parameters x and y.

```
public void DrawTo(int x, int y)
```

Parameters

`x` int

Integer which specifies a location on the horizontal axis.

`y` int

Integer which specifies a location on the vertical axis.

Exceptions

CanvasException

CanvasException is thrown when the given x and y values would result in drawing to a positon outside the boundary.

Load()

```
public void Load()
```

MoveTo(int, int)

A method to move to a (specified by parameters x and y) location, from the current X and Y position.

```
public void MoveTo(int x, int y)
```

Parameters

x int

Integer which specifies a location on the horizontal axis.

y int

Integer which specifies a location on the vertical axis.

Exceptions

CanvasException

CanvasException is thrown when the given x and y values would result in moving to a positon outside the boundary.

Rect(int, int, bool)

Method to draw a rectangle of specific width and height at the current X and Y position.

```
public void Rect(int width, int height, bool filled)
```

Parameters

`width` int

Integer which specifies the width of the rectangle.

`height` int

Integer which specifies the height of the rectangle.

`filled` bool

Boolean which specifies if the drawn rectangle should be filled.

Exceptions

CanvasException

CanvasException is thrown when the provided width or height is negative.

Reset()

Method to move the cursor to the starting position.

```
public void Reset()
```

Save()

```
public void Save()
```

Set(int, int)

A method used to initialize drawing components

```
public void Set(int width, int height)
```

Parameters

`width` int

Integer which specifies the width of the boundary.

`height` int

Integer which specifies the height of the boundary.

Remarks

The method sets a boundary the user can draw on, the boundary is set to the size of pictureBox. The method initializes the graphics object from the bitmap. The method creats a pen using the SetColour class and sets its colour to orange.

SetColour(Color)

An alternate SetColour method which takes in a Color type parameter to create a pen and a brush.

```
public void SetColour(Color Colour)
```

Parameters

`Colour` Color

Color which specifies a color we want to create a pen with.

Exceptions

CanvasException

CanvasException is thrown when the provided colour is not of the Color type.

SetColour(int, int, int)

Method to create a pen and a brush based on RGB values.

```
public void SetColour(int red, int green, int blue)
```

Parameters

`red` int

Integer which specifies the redness of the pen and brush.

`green` int

Integer which specifies the greenness of the pen and brush.

`blue` int

Integer which specifies the blueness of the pen and brush.

Remarks

The method takes 3 RGB values which are used to create a colour. The colour is then used to create a pen and a brush which the program uses to draw shapes.

Exceptions

CanvasException

CanvasException is thrown if any of the given RGB values are smaller than 0 or bigger than 255.

Tri(int, int)

Method to create a rectangle with a filled triangle inside of it.

```
public void Tri(int width, int height)
```

Parameters

`width` int

Integer which specifies the width of the rectangle.

`height` int

Integer which specifies the height of the rectanlge.

Exceptions

CanvasException

CanvasException is thrown when the provided width or height is negative.

WriteText(string)

Method to write text on the screen.

```
public void WriteText(string text)
```

Parameters

`text` string

String which is the text to be outputted on the screen.

getBitmap()

Method to receive the bitmap which contains the drawings.

```
public object getBitmap()
```

Returns

object

returns the bitmap

Exceptions

CanvasException

CanvasException is thrown if the bitmap is assigned to null.

Class Form1

Namespace: ASE_Assignment

Assembly: ASE Assignment.dll

Main class of the program. It contains the graphical elements of the interface that the user sees alongside methods that initialize and handle user input

```
public class Form1 : Form, IDropTarget, ISynchronizeInvoke, IWin32Window,
IBindableComponent, IComponent, IDisposable, IContainerControl
```

Inheritance

object ← MarshalByRefObject ← Component ← Control ← ScrollableControl ← ContainerControl ← Form ← Form1

Implements

IDropTarget, ISynchronizeInvoke, IWin32Window, IBindableComponent, IComponent, IDisposable, IContainerControl

Inherited Members

Form.SetVisibleCore(bool) , Form.Activate() , Form.ActivateMdiChild(Form) , Form.AddOwnedForm(Form) , Form.AdjustFormScrollbars(bool) , Form.Close() , Form.CreateAccessibilityInstance() , Form.CreateControlsInstance() , Form.CreateHandle() , Form.DefWndProc(ref Message) , Form.ProcessMnemonic(char) , Form.CenterToParent() , Form.CenterToScreen() , Form.LayoutMdi(MdiLayout) , Form.OnActivated(EventArgs) , Form.OnBackgroundImageChanged(EventArgs) , Form.OnBackgroundImageLayoutChanged(EventArgs) , Form.OnClosing(CancelEventArgs) , Form.OnClosed(EventArgs) , Form.OnFormClosing(FormClosingEventArgs) , Form.OnFormClosed(FormClosedEventArgs) , Form.OnCreateControl() , Form.OnDeactivate(EventArgs) , Form.OnEnabledChanged(EventArgs) , Form.OnEnter(EventArgs) , Form.OnFontChanged(EventArgs) , Form.OnHandleCreated(EventArgs) , Form.OnHandleDestroyed(EventArgs) , Form.OnHelpButtonClicked(CancelEventArgs) , Form.OnLayout(LayoutEventArgs) , Form.OnLoad(EventArgs) , Form.OnMaximizedBoundsChanged(EventArgs) , Form.OnMaximumSizeChanged(EventArgs) , Form.OnMinimumSizeChanged(EventArgs) , Form.OnInputLanguageChanged(InputLanguageChangedEventArgs) , Form.OnInputLanguageChanging(InputLanguageChangingEventArgs) , Form.OnVisibleChanged(EventArgs) , Form.OnMdiChildActivate(EventArgs) , Form.OnMenuStart(EventArgs) , Form.OnMenuComplete(EventArgs) , Form.OnPaint(PaintEventArgs) , Form.OnResize(EventArgs) ,

Form.OnDpiChanged(DpiChangedEventArgs) , Form.OnGetDpiScaledSize(int, int, ref Size) ,
Form.OnRightToLeftLayoutChanged(EventArgs) , Form.OnShown(EventArgs) ,
Form.OnTextChanged(EventArgs) , Form.ProcessCmdKey(ref Message, Keys) ,
Form.ProcessDialogKey(Keys) , Form.ProcessDialogChar(char) ,
Form.ProcessKeyPreview(ref Message) , Form.ProcessTabKey(bool) ,
Form.RemoveOwnedForm(Form) , Form.Select(bool, bool) ,
Form.GetScaledBounds(Rectangle, SizeF, BoundsSpecified) ,
Form.ScaleControl(SizeF, BoundsSpecified) , Form.SetBoundsCore(int, int, int, int, BoundsSpecified) ,
Form.SetClientSizeCore(int, int) , Form.SetDesktopBounds(int, int, int, int) ,
Form.SetDesktopLocation(int, int) , Form.Show(IWin32Window) , Form.ShowDialog() ,
Form.ShowDialog(IWin32Window) , Form.ToString() , Form.UpdateDefaultButton() ,
Form.OnResizeBegin(EventArgs) , Form.OnResizeEnd(EventArgs) ,
Form.OnStyleChanged(EventArgs) , Form.ValidateChildren() ,
Form.ValidateChildren(ValidationConstraints) , Form.WndProc(ref Message) , Form.AcceptButton ,
Form.ActiveForm , Form.ActiveMdiChild , Form.AllowTransparency , Form.AutoScroll ,
Form.AutoSize , Form.AutoSizeMode , Form.AutoValidate , Form.BackColor ,
Form.FormBorderStyle , Form.CancelButton , Form.ClientSize , Form.ControlBox ,
Form.CreateParams , Form.DefaultImeMode , Form.DefaultSize , Form.DesktopBounds ,
Form.DesktopLocation , Form.DialogResult , Form.HelpButton , Form.Icon , Form.IsMdiChild ,
Form.IsMdiContainer , Form.IsRestrictedWindow , Form.KeyPreview , Form.Location ,
Form.MaximizedBounds , Form.MaximumSize , Form.MainMenuStrip , Form.MinimumSize ,
Form.MaximizeBox , Form.MdiChildren , Form.MdiChildrenMinimizedAnchorBottom ,
Form.MdiParent , Form.MinimizeBox , Form.Modal , Form.Opacity , Form.OwnedForms ,
Form.Owner , Form.RestoreBounds , Form.RightToLeftLayout , Form.ShowInTaskbar ,
Form.ShowIcon , Form.ShowWithoutActivation , Form.Size , Form.SizeGripStyle ,
Form.StartPosition , Form.Text , Form.TopLevel , Form.TopMost , Form.TransparencyKey ,
Form.WindowState , Form.AutoSizeChanged , Form.AutoValidateChanged ,
Form.HelpButtonClicked , Form.MaximizedBoundsChanged , Form.MaximumSizeChanged ,
Form.MinimumSizeChanged , Form.Activated , Form.Deactivate , Form.FormClosing ,
Form.FormClosed , Form.Load , Form.MdiChildActivate , Form.MenuComplete ,
Form.MenuStart , Form.InputLanguageChanged , Form.InputLanguageChanging ,
Form.RightToLeftLayoutChanged , Form.Shown , Form.DpiChanged , Form.ResizeBegin ,
Form.ResizeEnd , ContainerControl.OnAutoValidateChanged(EventArgs) ,
ContainerControl.OnParentChanged(EventArgs) , ContainerControl.PerformAutoScale() ,
ContainerControl.RescaleConstantsForDpi(int, int) , ContainerControl.Validate() ,
ContainerControl.Validate(bool) , ContainerControl.AutoScaleDimensions ,
ContainerControl.AutoScaleFactor , ContainerControl.AutoScaleMode ,
ContainerControl.BindingContext , ContainerControl.CanEnableIme ,
ContainerControl.ActiveControl , ContainerControl.CurrentAutoScaleDimensions ,
ContainerControl.ParentForm , ScrollableControl.ScrollStateAutoScrolling ,

ScrollableControl.ScrollStateHScrollVisible , ScrollableControl.ScrollStateVScrollVisible ,
ScrollableControl.ScrollStateUserHasScrolled , ScrollableControl.ScrollStateFullDrag ,
ScrollableControl.GetScrollState(int) , ScrollableControl.OnMouseWheel(MouseEventArgs) ,
ScrollableControl.OnRightToLeftChanged(EventArgs) ,
ScrollableControl.OnPaintBackground(PaintEventArgs) ,
ScrollableControl.OnPaddingChanged(EventArgs) , ScrollableControl.SetDisplayRectLocation(int, int) ,
ScrollableControl.ScrollControlIntoView(Control) , ScrollableControl.ScrollToControl(Control) ,
ScrollableControl.OnScroll(ScrollEventArgs) , ScrollableControl.SetAutoScrollMargin(int, int) ,
ScrollableControl.SetScrollState(int, bool) , ScrollableControl.AutoScrollMargin ,
ScrollableControl.AutoScrollPosition , ScrollableControl.AutoScrollMinSize ,
ScrollableControl.DisplayRectangle , ScrollableControl.HScroll , ScrollableControl.HorizontalScroll ,
ScrollableControl.VScroll , ScrollableControl.VerticalScroll , ScrollableControl.Scroll ,
Control.GetAccessibilityObjectById(int) , Control.SetAutoSizeMode(AutoSizeMode) ,
Control.GetAutoSizeMode() , Control.GetPreferredSize(Size) ,
Control.AccessibilityNotifyClients(AccessibleEvents, int) ,
Control.AccessibilityNotifyClients(AccessibleEvents, int, int) , Control.BeginInvoke(Delegate) ,
Control.BeginInvoke(Action) , Control.BeginInvoke(Delegate, params object[]) ,
Control.BringToFront() , Control.Contains(Control) , Control.CreateGraphics() ,
Control.CreateControl() , Control.DestroyHandle() , Control.DoDragDrop(object, DragDropEffects) ,
Control.DrawToBitmap(Bitmap, Rectangle) , Control.EndInvoke(IAsyncResult) , Control.FindForm() ,
Control.GetTopLevel() , Control.RaiseKeyEvent(object, KeyEventArgs) ,
Control.RaiseMouseEvent(object, MouseEventArgs) , Control.Focus() ,
Control.FromChildHandle(IntPtr) , Control.FromHandle(IntPtr) ,
Control.GetChildAtPoint(Point, GetChildAtPointSkip) , Control.GetChildAtPoint(Point) ,
Control.GetContainerControl() , Control.GetNextControl(Control, bool) ,
Control.GetStyle(ControlStyles) , Control.Hide() , Control.InitLayout() , Control.Invalidate(Region) ,
Control.Invalidate(Region, bool) , Control.Invalidate() , Control.Invalidate(bool) ,
Control.Invalidate(Rectangle) , Control.Invalidate(Rectangle, bool) , Control.Invoke(Action) ,
Control.Invoke(Delegate) , Control.Invoke(Delegate, params object[]) ,
Control.Invoke<T>(Func<T>) , Control.InvokePaint(Control, PaintEventArgs) ,
Control.InvokePaintBackground(Control, PaintEventArgs) , Control.IsKeyLocked(Keys) ,
Control.IsInputChar(char) , Control.IsInputKey(Keys) , Control.IsMnemonic(char, string) ,
Control.LogicalToDeviceUnits(int) , Control.LogicalToDeviceUnits(Size) ,
Control.ScaleBitmapLogicalToDevice(ref Bitmap) , Control.NotifyInvalidate(Rectangle) ,
Control.InvokeOnClick(Control, EventArgs) , Control.OnAutoSizeChanged(EventArgs) ,
Control.OnBackColorChanged(EventArgs) , Control.OnBindingContextChanged(EventArgs) ,
Control.OnCausesValidationChanged(EventArgs) , Control.OnContextMenuStripChanged(EventArgs) ,
Control.OnCursorChanged(EventArgs) , Control.OnDockChanged(EventArgs) ,
Control.OnForeColorChanged(EventArgs) , Control.OnNotifyMessage(Message) ,
Control.OnParentBackColorChanged(EventArgs) ,

Control.OnParentBackgroundImageChanged(EventArgs) ,
Control.OnParentBindingContextChanged(EventArgs) , Control.OnParentCursorChanged(EventArgs) ,
Control.OnParentEnabledChanged(EventArgs) , Control.OnParentFontChanged(EventArgs) ,
Control.OnParentForeColorChanged(EventArgs) , Control.OnParentRightToLeftChanged(EventArgs) ,
Control.OnParentVisibleChanged(EventArgs) , Control.OnPrint(PaintEventArgs) ,
Control.OnTabIndexChanged(EventArgs) , Control.OnTabStopChanged(EventArgs) ,
Control.OnClick(EventArgs) , Control.OnClientSizeChanged(EventArgs) ,
Control.OnControlAdded(ControlEventArgs) , Control.OnControlRemoved(ControlEventArgs) ,
Control.OnLocationChanged(EventArgs) , Control.OnDoubleClick(EventArgs) ,
Control.OnDragEnter(DragEventArgs) , Control.OnDragOver(DragEventArgs) ,
Control.OnDragLeave(EventArgs) , Control.OnDragDrop(DragEventArgs) ,
Control.OnGiveFeedback(GiveFeedbackEventArgs) , Control.InvokeGotFocus(Control, EventArgs) ,
Control.OnGotFocus(EventArgs) , Control.OnHelpRequested(HelpEventArgs) ,
Control.OnInvalidated(InvalidateEventArgs) , Control.OnKeyDown(KeyEventArgs) ,
Control.OnKeyPress(KeyPressEventArgs) , Control.OnKeyUp(KeyEventArgs) ,
Control.OnLeave(EventArgs) , Control.InvokeLostFocus(Control, EventArgs) ,
Control.OnLostFocus(EventArgs) , Control.OnMarginChanged(EventArgs) ,
Control.OnMouseDoubleClick(MouseEventArgs) , Control.OnMouseClick(MouseEventArgs) ,
Control.OnMouseCaptureChanged(EventArgs) , Control.OnMouseDown(MouseEventArgs) ,
Control.OnMouseEnter(EventArgs) , Control.OnMouseLeave(EventArgs) ,
Control.OnDpiChangedBeforeParent(EventArgs) , Control.OnDpiChangedAfterParent(EventArgs) ,
Control.OnMouseHover(EventArgs) , Control.OnMouseMove(MouseEventArgs) ,
Control.OnMouseUp(MouseEventArgs) , Control.OnMove(EventArgs) ,
Control.OnQueryContinueDrag(QueryContinueDragEventArgs) ,
Control.OnRegionChanged(EventArgs) , Control.OnPreviewKeyDown(PreviewKeyDownEventArgs) ,
Control.OnSizeChanged(EventArgs) , Control.OnChangeUICues(UICuesEventArgs) ,
Control.OnSystemColorsChanged(EventArgs) , Control.OnValidating(CancelEventArgs) ,
Control.OnValidated(EventArgs) , Control.PerformLayout() , Control.PerformLayout(Control, string) ,
Control.PointToClient(Point) , Control.PointToScreen(Point) ,
Control.PreProcessMessage(ref Message) , Control.PreProcessControlMessage(ref Message) ,
Control.ProcessKeyEventArgs(ref Message) , Control.ProcessKeyMessage(ref Message) ,
Control.RaiseDragEvent(object, DragEventArgs) , Control.RaisePaintEvent(object, PaintEventArgs) ,
Control.RecreateHandle() , Control.RectangleToClient(Rectangle) ,
Control.RectangleToScreen(Rectangle) , Control.ReflectMessage(IntPtr, ref Message) ,
Control.Refresh() , Control.ResetMouseEventArgs() , Control.ResetText() , Control.ResumeLayout() ,
Control.ResumeLayout(bool) , Control.Scale(SizeF) , Control.Select() ,
Control.SelectNextControl(Control, bool, bool, bool, bool) , Control.SendToBack() ,
Control.SetBounds(int, int, int, int) , Control.SetBounds(int, int, int, int, BoundsSpecified) ,
Control.SizeFromClientSize(Size) , Control.SetStyle(ControlStyles, bool) , Control.SetTopLevel(bool) ,
Control.RtlTranslateAlignment(HorizontalAlignment) ,

Control.RtlTranslateAlignment(LeftRightAlignment) ,
Control.RtlTranslateAlignment(ContentAlignment) ,
Control.RtlTranslateHorizontal(HorizontalAlignment) ,
Control.RtlTranslateLeftRight(LeftRightAlignment) , Control.RtlTranslateContent(ContentAlignment) ,
Control.Show() , Control.SuspendLayout() , Control.Update() , Control.UpdateBounds() ,
Control.UpdateBounds(int, int, int, int) , Control.UpdateBounds(int, int, int, int, int, int) ,
Control.UpdateZOrder() , Control.UpdateStyles() , Control.OnImeModeChanged(EventArgs) ,
Control.AccessibilityObject , Control.AccessibleDefaultActionDescription ,
Control.AccessibleDescription , Control.AccessibleName , Control.AccessibleRole ,
Control.AllowDrop , Control.Anchor , Control.AutoScrollOffset , Control.LayoutEngine ,
Control.BackgroundImage , Control.BackgroundImageLayout , Control.Bottom , Control.Bounds ,
Control.CanFocus , Control.CanRaiseEvents , Control.CanSelect , Control.Capture ,
Control.CausesValidation , Control.CheckForIllegalCrossThreadCalls , Control.ClientRectangle ,
Control.CompanyName , Control.ContainsFocus , Control.ContextMenuStrip , Control.Controls ,
Control.Created , Control.Cursor , Control.DataBindings , Control.DefaultBackColor ,
Control.DefaultCursor , Control.DefaultFont , Control.DefaultForeColor , Control.DefaultMargin ,
Control.DefaultMaximumSize , Control.DefaultMinimumSize , Control.DefaultPadding ,
Control.DeviceDpi , Control.IsDisposed , Control.Disposing , Control.Dock ,
Control.DoubleBuffered , Control.Enabled , Control.Focused , Control.Font ,
Control.FontHeight , Control.ForeColor , Control.Handle , Control.HasChildren , Control.Height ,
Control.IsHandleCreated , Control.InvokeRequired , Control.IsAccessible ,
Control.IsAncestorSiteInDesignMode , Control.IsMirrored , Control.Left , Control.Margin ,
Control.ModifierKeys , Control.MouseButtons , Control.MousePosition , Control.Name ,
Control.Parent , Control.ProductName , Control.ProductVersion , Control.RecreatingHandle ,
Control.Region , Control.RenderRightToLeft , Control.ResizeRedraw , Control.Right ,
Control.RightToLeft , Control.ScaleChildren , Control.Site , Control.TabIndex , Control.TabStop ,
Control.Tag , Control.Top , Control.TopLevelControl , Control.ShowKeyboardCues ,
Control.ShowFocusCues , Control.UseWaitCursor , Control.Visible , Control.Width ,
Control.PreferredSize , Control.Padding , Control.ImeMode , Control.ImeModeBase ,
Control.PropagatingImeMode , Control.BackColorChanged , Control.BackgroundImageChanged ,
Control.BackgroundImageLayoutChanged , Control.BindingContextChanged ,
Control.CausesValidationChanged , Control.ClientSizeChanged ,
Control.ContextMenuStripChanged , Control.CursorChanged , Control.DockChanged ,
Control.EnabledChanged , Control.FontChanged , Control.ForeColorChanged ,
Control.LocationChanged , Control.MarginChanged , Control.RegionChanged ,
Control.RightToLeftChanged , Control.SizeChanged , Control.TabIndexChanged ,
Control.TabStopChanged , Control.TextChanged , Control.VisibleChanged , Control.Click ,
Control.ControlAdded , Control.ControlRemoved , Control.DragDrop , Control.DragEnter ,
Control.DragOver , Control.DragLeave , Control.GiveFeedback , Control.HandleCreated ,
Control.HandleDestroyed , Control.HelpRequested , Control.Invalidated ,

Control.PaddingChanged , Control.Paint , Control.QueryContinueDrag , Control.QueryAccessibilityHelp , Control.DoubleClick , Control.Enter , Control.GotFocus , Control.KeyDown , Control.KeyPress , Control.KeyUp , Control.Layout , Control.Leave , Control.LostFocus , Control.MouseClick , Control.MouseDoubleClick , Control.MouseCaptureChanged , Control.MouseDown , Control.MouseEnter , Control.MouseLeave , Control.DpiChangedBeforeParent , Control.DpiChangedAfterParent , Control.MouseHover , Control.MouseMove , Control.MouseUp , Control.MouseWheel , Control.Move , Control.PreviewKeyDown , Control.Resize , Control.ChangeUICues , Control.StyleChanged , Control.SystemColorsChanged , Control.Validating , Control.Validated , Control.ParentChanged , Control.ImeModeChanged , Component.Dispose() , Component.GetService(Type) , Component.Container , Component.DesignMode , Component.Events , Component.Disposed , MarshalByRefObject.GetLifetimeService() , MarshalByRefObject.InitializeLifetimeService() , MarshalByRefObject.MemberwiseClone(bool) , object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.MemberwiseClone() , object.ReferenceEquals(object, object)

Constructors

Form1()

Constructor for the Form1 class, its used to initialize all elements necessary to to view, draw and take input from the user

```
public Form1()
```

Methods

Dispose(bool)

Clean up any resources being used.

```
protected override void Dispose(bool disposing)
```

Parameters

`disposing` bool

true if managed resources should be disposed; otherwise, false.

Class ownArray

Namespace: ASE Assignment

Assembly: ASE Assignment.dll

Class that implements array functionality

```
public class ownArray : Evaluation, ICommand
```

Inheritance

object ← Command ← Evaluation ← ownArray

Implements

ICommand

Derived

ownPeek, ownPoke

Inherited Members

Evaluation.expression , Evaluation.evaluatedExpression , Evaluation.varName , Evaluation.value , Evaluation.ProcessExpression(string) , Evaluation.Expression , Evaluation.VarName , Evaluation.Value , Evaluation.Local , Command.program , Command.parameterList , Command.parameters , Command.paramsint , Command.Set(StoredProgram, string) , Command.ProcessParameters(string) , Command.ToString() , Command.Program , Command.Name , Command.ParameterList , Command.Parameters , Command.Paramsint , object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.MemberwiseClone() , object.ReferenceEquals(object, object)

Constructors

ownArray()

basic constructor

```
public ownArray()
```

Fields

column

specific column to access/set array value

```
protected int column
```

Field Value

int

columnString

Expression column

```
protected string columnString
```

Field Value

string

columns

size of columns in array

```
protected int columns
```

Field Value

int

intArray

the integer array to be set

```
protected int[,] intArray
```

Field Value

int[,]

peekVar

the variable used to recieve the value from the array

```
protected string peekVar
```

Field Value

string

pokeValue

the value to set at an array index

```
protected string pokeValue
```

Field Value

string

realArray

the double array to be set

```
protected double[,] realArray
```

Field Value

double[,]

row

the specific row used to access/set a value in array

```
protected int row
```

Field Value

int

rowString

expression row

```
protected string rowString
```

Field Value

string

rows

the size of rows in array

```
protected int rows
```

Field Value

int

type

integer or double

```
protected string type
```

Field Value

string

valueInt

the value recieved from the integer array at specific index

```
protected int valueInt
```

Field Value

int

valueReal

the value recieved from the real array at specific index

```
protected double valueReal
```

Field Value

double

Properties

Columns

get the column size of the array

```
protected int Columns { get; }
```

Property Value

int

Rows

get the row size of the array

```
protected int Rows { get; }
```

Property Value

int

Methods

CheckParameters(string[])

checks to see if the array is correctly defined

```
public override void CheckParameters(string[] parameterList)
```

Parameters

`parameterList` string[]

the parameters used to set an array

Exceptions

BOOSEException

thrown when the array is not declared correctly

Compile()

Adds the array to the variable list and sets its type.

```
public override void Compile()
```

Exceptions

BOOSEException

Thrown when the array is not of type integer or real

Execute()

creates array using values from the compiliation.

```
public override void Execute()
```

Exceptions

BOOSEException

Thrown when the array is not of type int or real

GetIntArray(int, int)

returns the value at specific index

```
public int GetIntArray(int row, int col)
```

Parameters

`row` int

row index

`col` int

column index

Returns

int

the value at index

Exceptions

CommandException

thrown when the passed row and column are out of the array bounds

GetRealArray(int, int)

returns the value at a specific index

```
public double GetRealArray(int row, int col)
```

Parameters

`row` int

row index

`col` int

column index

Returns

double

the value at index

Exceptions

CommandException

thrown when the passed row and column are out of the array bounds

ProcessArrayParametersExecute(bool)

execute method for peek and poke, sets or retrieves a value from the array

```
protected void ProcessArrayParametersExecute(bool peekOrPoke)
```

Parameters

`peekOrPoke` bool

if true, peek called this method, otherwise poke did

Exceptions

CommandException

thrown when the array being accessed does not exist

ProcessArrayParamteresCompile(bool)

compile method for peek and poke, processes the parameters passed in those commands

```
protected void ProcessArrayParamteresCompile(bool peekOrPoke)
```

Parameters

`peekOrPoke` bool

if true, peek called this method, otherwise poke did

Exceptions

BOOSEException

thrown when the array being accessed does not exist

SetIntArray(int, int, int)

sets a value in a int array at specific index

```
public void SetIntArray(int val, int row, int col)
```

Parameters

`val` int

value to be set at index

`row` int

row index

`col` int

column index

Exceptions

CommandException

thrown when row and column given are out of bounds of the array

SetRealArray(double, int, int)

sets a value in a real array at specific index

```
public void SetRealArray(double val, int row, int col)
```

Parameters

`val` double

value to be set at index

`row` int

row index

`col` int

column index

Exceptions

CommandException

thrown when row and column given are out of bounds of the array

Class ownCommandfactory

Namespace: ASE_Assignment

Assembly: ASE Assignment.dll

The class extends CommandFactory from BOOSE. Both classes receive commands from the user as input and create new objects of their corresponding classes, if they exist in the factory.

```
public class ownCommandfactory : CommandFactory, ICommandFactory
```

Inheritance

object ← CommandFactory ← ownCommandfactory

Implements

ICommandFactory

Inherited Members

object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.MemberwiseClone() , object.ReferenceEquals(object, object) , object.ToString()

Methods

MakeCommand(string)

Method to see if a command exists, if it does; it creates a new object of the corresponding command.

```
public override ICommand MakeCommand(string commandType)
```

Parameters

`commandType` string

String which is the command passed in by the Parser after user inputs their text.

Returns

ICommand

Class ownCompoundCommand

Namespace: ASE_Assignment

Assembly: ASE Assignment.dll

class used to find the types of command

```
public class ownCompoundCommand : ConditionalCommand, ICommand
```

Inheritance

object ← Command ← Evaluation ← Boolean ← ConditionalCommand ← ownCompoundCommand

Implements

ICommand

Derived

ownElse, ownEnd, ownIf, ownWhile

Inherited Members

ConditionalCommand.endLineNumber , ConditionalCommand.Compile() , ConditionalCommand.Execute() , ConditionalCommand.EndLineNumber , ConditionalCommand.Condition , ConditionalCommand.LineNumber , ConditionalCommand.CondType , ConditionalCommand.ReturnLineNumber , Boolean.Restrictions() , Boolean.BoolValue , Evaluation.expression , Evaluation.evaluatedExpression , Evaluation.varName , Evaluation.value , Evaluation.CheckParameters(string[]) , Evaluation.ProcessExpression(string) , Evaluation.Expression , Evaluation.VarName , Evaluation.Value , Evaluation.Local , Command.program , Command.parameterList , Command.parameters , Command.paramsint , Command.Set(StoredProgram, string) , Command.ProcessParameters(string) , Command.ToString() , Command.Program , Command.Name , Command.ParameterList , Command.Parameters , Command.Paramsint , object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.MemberwiseClone() , object.ReferenceEquals(object, object)

Properties

CoresspondingCommand

returns the type a command is.

```
public ConditionalCommand CoresspondingCommand { get; set; }
```

Property Value

ConditionalCommand

Class ownElse

Namespace: ASE_Assignment

Assembly: ASE Assignment.dll

class responsible for elses in the program when if conditions return false

```
public class ownElse : ownCompoundCommand, ICommand
```

Inheritance

object ← Command ← Evaluation ← Boolean ← ConditionalCommand ← ownCompoundCommand ← ownElse

Implements

ICommand

Inherited Members

ownCompoundCommand.CoressspondingCommand , ConditionalCommand.endLineNumber , ConditionalCommand.EndLineNumber , ConditionalCommand.Condition , ConditionalCommand.LineNumber , ConditionalCommand.CondType , ConditionalCommand.ReturnLineNumber , Boolean.Restrictions() , Boolean.BoolValue , Evaluation.expression , Evaluation.evaluatedExpression , Evaluation.varName , Evaluation.value , Evaluation.CheckParameters(string[]) , Evaluation.ProcessExpression(string) , Evaluation.Expression , Evaluation.VarName , Evaluation.Value , Evaluation.Local , Command.program , Command.parameterList , Command.parameters , Command.paramsint , Command.Set(StoredProgram, string) , Command.ProcessParameters(string) , Command.ToString() , Command.Program , Command.Name , Command.ParameterList , Command.Parameters , Command.Paramsint , object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.MemberwiseClone() , object.ReferenceEquals(object, object)

Methods

Compile()

retrieves the most recent if statement and sets its endline number. pushes itself on the stack in its place

```
public override void Compile()
```

Execute()

checks to see if its condition is true, then goes to the according place in the program.

```
public override void Execute()
```

Class ownEnd

Namespace: ASE_Assignment

Assembly: ASE Assignment.dll

class which is reponsible for ending if, while and for blocks of code

```
public class ownEnd : ownCompoundCommand, ICommand
```

Inheritance

object ← Command ← Evaluation ← Boolean ← ConditionalCommand ← ownCompoundCommand ← ownEnd

Implements

ICommand

Inherited Members

ownCompoundCommand.CoressspondingCommand , ConditionalCommand.endLineNumber , ConditionalCommand.EndLineNumber , ConditionalCommand.Condition , ConditionalCommand.LineNumber , ConditionalCommand.CondType , ConditionalCommand.ReturnLineNumber , Boolean.Restrictions() , Boolean.BoolValue , Evaluation.expression , Evaluation.evaluatedExpression , Evaluation.varName , Evaluation.value , Evaluation.CheckParameters(string[]) , Evaluation.ProcessExpression(string) , Evaluation.Expression , Evaluation.VarName , Evaluation.Value , Evaluation.Local , Command.program , Command.parameterList , Command.parameters , Command.paramsint , Command.Set(StoredProgram, string) , Command.ProcessParameters(string) , Command.ToString() , Command.Program , Command.Name , Command.ParameterList , Command.Parameters , Command.Paramsint , object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.MemberwiseClone() , object.ReferenceEquals(object, object)

Constructors

ownEnd()

basic constructor

```
public ownEnd()
```

Methods

Compile()

checks to see if the end is reponsible for while, if or for and sets the start and end number lines for those commands

```
public override void Compile()
```

Exceptions

CommandException

Execute()

checks to see if while or for should be repeated or if they finished their iterations.

```
public override void Execute()
```

Exceptions

CommandException

Class ownFor

Namespace: ASE_Assignment

Assembly: ASE Assignment.dll

class which allows for loops in the application

```
public class ownFor : ConditionalCommand, ICommand
```

Inheritance

object ← Command ← Evaluation ← Boolean ← ConditionalCommand ← ownFor

Implements

ICommand

Inherited Members

ConditionalCommand.endLineNumber , ConditionalCommand.EndLineNumber , ConditionalCommand.Condition , ConditionalCommand.LineNumber , ConditionalCommand.CondType , ConditionalCommand.ReturnLineNumber , Boolean.Restrictions() , Boolean.BoolValue , Evaluation.expression , Evaluation.evaluatedExpression , Evaluation.varName , Evaluation.value , Evaluation.CheckParameters(string[]) , Evaluation.ProcessExpression(string) , Evaluation.Expression , Evaluation.VarName , Evaluation.Value , Evaluation.Local , Command.program , Command.parameterList , Command.parameters , Command.paramsint , Command.Set(StoredProgram, string) , Command.ProcessParameters(string) , Command.ToString() , Command.Program , Command.Name , Command.ParameterList , Command.Parameters , Command.Paramsint , object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.MemberwiseClone() , object.ReferenceEquals(object, object)

Properties

End

the value at which the loop ends

```
public int End { get; set; }
```

Property Value

int

Start

the value from which the loop begins

```
public int Start { get; set; }
```

Property Value

int

Steps

determines how big the increment is

```
public int Steps { get; set; }
```

Property Value

int

Var

used to retrieve the variable used in the for expression

```
public Evaluation Var { get; }
```

Property Value

Evaluation

Varname

retrieve the variable name of the variable used in the expression

```
public string Varname { get; }
```

Property Value

string

Methods

Compile()

processes parameters and the expression used by the loop

```
public override void Compile()
```

Exceptions

CommandException

thrown when the parameters are invalid

Execute()

attempts to convert the start, end and steps values into integers.

```
public override void Execute()
```

Class ownIf

Namespace: ASE_Assignment

Assembly: ASE Assignment.dll

class that enables if conditions in the program

```
public class ownIf : ownCompoundCommand, ICommand
```

Inheritance
object ← Command ← Evaluation ← Boolean ← ConditionalCommand ← ownCompoundCommand ← ownIf

Implements
ICommand

Inherited Members
ownCompoundCommand.CoressspondingCommand , ConditionalCommand.endLineNumber , ConditionalCommand.EndLineNumber , ConditionalCommand.Condition , ConditionalCommand.LineNumber , ConditionalCommand.CondType , ConditionalCommand.ReturnLineNumber , Boolean.Restrictions() , Boolean.BoolValue , Evaluation.expression , Evaluation.evaluatedExpression , Evaluation.varName , Evaluation.value , Evaluation.CheckParameters(string[]) , Evaluation.ProcessExpression(string) , Evaluation.Expression , Evaluation.VarName , Evaluation.Value , Evaluation.Local , Command.program , Command.parameterList , Command.parameters , Command.paramsint , Command.Set(StoredProgram, string) , Command.ProcessParameters(string) , Command.ToString() , Command.Program , Command.Name , Command.ParameterList , Command.Parameters , Command.Paramsint , object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.MemberwiseClone() , object.ReferenceEquals(object, object)

Methods

Compile()

processes parameters

```
public override void Compile()
```

Execute()

checks to see if the passed expression is true or false, then moves to the appriopriate place in code

```
public override void Execute()
```

Class ownInt

Namespace: ASE_Assignment

Assembly: ASE Assignment.dll

Class used to handle integer variables.

```
public class ownInt : Evaluation, ICommand
```

Inheritance

object ← Command ← Evaluation ← ownInt

Implements

ICommand

Inherited Members

Evaluation.expression , Evaluation.evaluatedExpression , Evaluation.varName , Evaluation.value , Evaluation.CheckParameters(string[]) , Evaluation.ProcessExpression(string) , Evaluation.Expression , Evaluation.VarName , Evaluation.Value , Evaluation.Local , Command.program , Command.parameterList , Command.parameters , Command.paramsint , Command.Set(StoredProgram, string) , Command.ProcessParameters(string) , Command.ToString() , Command.Program , Command.Name , Command.ParameterList , Command.Parameters , Command.Paramsint , object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.MemberwiseClone() , object.ReferenceEquals(object, object)

Constructors

ownInt()

```
public ownInt()
```

Methods

Compile()

Calls the parent compile method and adds the command to the storedProgram variable list

```
public override void Compile()
```

Execute()

Calls the parent execute method, and checks to see if the expression evaluated by the compile method is an integer. If it is an integer, it updates the variable which was added to the storedProgram variable list using the Compile() method to have the evaluated value that was acquired in the parent class.

```
public override void Execute()
```

Class ownParser

Namespace: ASE Assignment

Assembly: ASE Assignment.dll

```
public class ownParser : IParser
```

Inheritance

object ← ownParser

Implements

IParser

Inherited Members

object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.MemberwiseClone() , object.ReferenceEquals(object, object) , object.ToString()

Constructors

ownParser(CommandFactory, StoredProgram)

Constructor that sets

```
public ownParser(CommandFactory factory, StoredProgram program)
```

Parameters

`factory` CommandFactory

`program` StoredProgram

Methods

AddSpaces(string)

Looks for symbols in a line from the program to add spaces around them. This is done so the ParseCommand method is able to split the line into words

```
public string AddSpaces(string text)
```

Parameters

`text` string

Line from the program to be formatted

Returns

string

The formatted string

ParseCommand(string)

Splits up user input from lines to words to differentiate commands, types, variables, methods and parameters.

```
public ICommand ParseCommand(string Line)
```

Parameters

`Line` string

Line passed by ParseProgram

Returns

ICommand

A command with paramaters

ParseProgram(string)

Splits up user input from the entire program into lines, which are passed int the ParseCommand method

```
public void ParseProgram(string program)
```

Parameters

`program` string

Full user created program

Class ownPeek

Namespace: ASE_Assignment

Assembly: ASE Assignment.dll

class used to access elements from an array and return their values

```
public class ownPeek : ownArray, ICommand
```

Inheritance

object ← Command ← Evaluation ← ownArray ← ownPeek

Implements

ICommand

Inherited Members

ownArray.column , ownArray.columnString , ownArray.columns , ownArray.intArray , ownArray.peekVar , ownArray.pokeValue , ownArray.realArray , ownArray.row , ownArray.rowString , ownArray.rows , ownArray.type , ownArray.valueInt , ownArray.valueReal , ownArray.Columns , ownArray.Rows , ownArray.CheckParameters(string[]) , ownArray.GetIntArray(int, int) , ownArray.GetRealArray(int, int) , ownArray.ProcessArrayParamteresCompile(bool) , ownArray.ProcessArrayParametersExecute(bool) , ownArray.SetIntArray(int, int, int) , ownArray.SetRealArray(double, int, int) , Evaluation.expression , Evaluation.evaluatedExpression , Evaluation.varName , Evaluation.value , Evaluation.ProcessExpression(string) , Evaluation.Expression , Evaluation.VarName , Evaluation.Value , Evaluation.Local , Command.program , Command.parameterList , Command.parameters , Command.paramsint , Command.Set(StoredProgram, string) , Command.ProcessParameters(string) , Command.ToString() , Command.Program , Command.Name , Command.ParameterList , Command.Parameters , Command.Paramsint , object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.MemberwiseClone() , object.ReferenceEquals(object, object)

Methods

Compile()

calls the ProcessArrayParamteresCompile from its parent

```
public override void Compile()
```

Execute()

calls the ProcessArrayParamteresCompile from its parent and sets the value of a paremeter to the returned value

```
public override void Execute()
```

Exceptions

CommandException

Class ownPenColour

Namespace: ASE Assignment

Assembly: ASE Assignment.dll

Class used to create a pen of a specified colour

```
public class ownPenColour : CommandThreeParameters, ICommand
```

Inheritance

object ← Command ← CanvasCommand ← CommandOneParameter ← CommandTwoParameters ← CommandThreeParameters ← ownPenColour

Implements

ICommand

Inherited Members

CommandThreeParameters.param3 , CommandThreeParameters.param3unprocessed , CommandThreeParameters.CheckParameters(string[]) , CommandTwoParameters.param2 , CommandTwoParameters.param2unprocessed , CommandOneParameter.param1 , CommandOneParameter.param1unprocessed , CanvasCommand.yPos , CanvasCommand.xPos , CanvasCommand.canvas , CanvasCommand.Canvas , Command.program , Command.parameterList , Command.parameters , Command.paramsint , Command.Set(StoredProgram, string) , Command.Compile() , Command.ProcessParameters(string) , Command.ToString() , Command.Program , Command.Name , Command.ParameterList , Command.Parameters , Command.Paramsint , object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.MemberwiseClone() , object.ReferenceEquals(object, object)

Methods

Execute()

Method used to create a pen with a specific RGB colour by calling the Super class version of the Execute() command, alongside calling the SetColour by using the Canvas getter which will return the Draws class.

```
public override void Execute()
```

Class ownPoke

Namespace: ASE_Assignment

Assembly: ASE Assignment.dll

class used to set values at specific index in an array

```
public class ownPoke : ownArray, ICommand
```

Inheritance

object ← Command ← Evaluation ← ownArray ← ownPoke

Implements

ICommand

Inherited Members

ownArray.column , ownArray.columnString , ownArray.columns , ownArray.intArray , ownArray.peekVar , ownArray.pokeValue , ownArray.realArray , ownArray.row , ownArray.rowString , ownArray.rows , ownArray.type , ownArray.valueInt , ownArray.valueReal , ownArray.Columns , ownArray.Rows , ownArray.GetIntArray(int, int) , ownArray.GetRealArray(int, int) , ownArray.ProcessArrayParamteresCompile(bool) , ownArray.ProcessArrayParametersExecute(bool) , ownArray.SetIntArray(int, int, int) , ownArray.SetRealArray(double, int, int) , Evaluation.expression , Evaluation.evaluatedExpression , Evaluation.varName , Evaluation.value , Evaluation.ProcessExpression(string) , Evaluation.Expression , Evaluation.VarName , Evaluation.Value , Evaluation.Local , Command.program , Command.parameterList , Command.parameters , Command.paramsint , Command.Set(StoredProgram, string) , Command.ProcessParameters(string) , Command.ToString() , Command.Program , Command.Name , Command.ParameterList , Command.Parameters , Command.Paramsint , object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.MemberwiseClone() , object.ReferenceEquals(object, object)

Methods

CheckParameters(string[])

checks to see if there are at most 2 parameters associated with the command

```
public override void CheckParameters(string[] parameter)
```

Parameters

`parameter` string[]

a list of parameters

Exceptions

CommandException

thrown when the number of parameters is not 2 or 3

Compile()

calls the ProcessArrayParameterCompile method from its parent

```
public override void Compile()
```

Execute()

calls the ProcessArrayParametersExecute method from its parent

```
public override void Execute()
```

Class ownReal

Namespace: ASE_Assignment

Assembly: ASE Assignment.dll

```
public class ownReal : Evaluation, ICommand
```

Inheritance

object ← Command ← Evaluation ← ownReal

Implements

ICommand

Derived

ownSqrt

Inherited Members

Evaluation.expression , Evaluation.evaluatedExpression , Evaluation.varName , Evaluation.value , Evaluation.CheckParameters(string[]) , Evaluation.ProcessExpression(string) , Evaluation.Expression , Evaluation.VarName , Evaluation.Local , Command.program , Command.parameterList , Command.parameters , Command.paramsint , Command.Set(StoredProgram, string) , Command.ProcessParameters(string) , Command.ToString() , Command.Program , Command.Name , Command.ParameterList , Command.Parameters , Command.Paramsint , object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.MemberwiseClone() , object.ReferenceEquals(object, object)

Constructors

ownReal()

constructor which sets cultural information so decimal points can be used for double type values

```
public ownReal()
```

Properties

Value

the value of the variable

```
public double Value { get; set; }
```

Property Value

double

Methods

Compile()

processes parameters and adds the variable to the variable list

```
public override void Compile()
```

Execute()

attempts to evaluate expression and sets the variable value to a double

```
public override void Execute()
```

Exceptions

StoredProgramException

thrown when the value could not be processed as a double

Class ownSqrt

Namespace: ASE_Assignment

Assembly: ASE Assignment.dll

class that allows users to square root a number

```
public class ownSqrt : ownReal, ICommand
```

Inheritance

object ← Command ← Evaluation ← ownReal ← ownSqrt

Implements

ICommand

Inherited Members

ownReal.Value , Evaluation.expression , Evaluation.evaluatedExpression , Evaluation.varName , Evaluation.value , Evaluation.CheckParameters(string[]) , Evaluation.ProcessExpression(string) , Evaluation.Expression , Evaluation.VarName , Evaluation.Local , Command.program , Command.parameterList , Command.parameters , Command.paramsint , Command.Set(StoredProgram, string) , Command.ProcessParameters(string) , Command.ToString() , Command.Program , Command.Name , Command.ParameterList , Command.Parameters , Command.Paramsint , object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.MemberwiseClone() , object.ReferenceEquals(object, object)

Methods

Compile()

processes parameters and adds variable to variable list

```
public override void Compile()
```

Execute()

evaluates expression and square roots it.

```
public override void Execute()
```

Exceptions

CommandException

thrown when expression can't be processed

Class ownStoredProgram

Namespace: ASE_Assignment

Assembly: ASE Assignment.dll

```
public class ownStoredProgram : StoredProgram, IList, ICollection, IEnumerable,
ICloneable, IStoredProgram
```

Inheritance

object ← ArrayList ← StoredProgram ← ownStoredProgram

Implements

IList, ICollection, IEnumerable, ICloneable, IStoredProgram

Inherited Members

StoredProgram.SyntaxOk , StoredProgram.AddMethod(Method) , StoredProgram.GetMethod(string) , StoredProgram.GetVariable(int) , StoredProgram.FindVariable(Evaluation) , StoredProgram.DeleteVariable(string) , StoredProgram.IsExpression(string) , StoredProgram.EvaluateExpressionWithString(string) , StoredProgram.Push(ConditionalCommand) , StoredProgram.Pop() , StoredProgram.Add(Command) , StoredProgram.NextCommand() , StoredProgram.Commandsleft() , ArrayList.Adapter(IList) , ArrayList.Add(object) , ArrayList.AddRange(ICollection) , ArrayList.BinarySearch(int, int, object, IComparer) , ArrayList.BinarySearch(object) , ArrayList.BinarySearch(object, IComparer) , ArrayList.Clear() , ArrayList.Clone() , ArrayList.Contains(object) , ArrayList.CopyTo(Array) , ArrayList.CopyTo(Array, int) , ArrayList.CopyTo(int, Array, int, int) , ArrayList.FixedSize(ArrayList) , ArrayList.FixedSize(IList) , ArrayList.GetEnumerator() , ArrayList.GetEnumerator(int, int) , ArrayList.GetRange(int, int) , ArrayList.IndexOf(object) , ArrayList.IndexOf(object, int) , ArrayList.IndexOf(object, int, int) , ArrayList.Insert(int, object) , ArrayList.InsertRange(int, ICollection) , ArrayList.LastIndexOf(object) , ArrayList.LastIndexOf(object, int) , ArrayList.LastIndexOf(object, int, int) , ArrayList.ReadOnly(ArrayList) , ArrayList.ReadOnly(IList) , ArrayList.Remove(object) , ArrayList.RemoveAt(int) , ArrayList.RemoveRange(int, int) , ArrayList.Repeat(object, int) , ArrayList.Reverse() , ArrayList.Reverse(int, int) , ArrayList.SetRange(int, ICollection) , ArrayList.Sort() , ArrayList.Sort(IComparer) , ArrayList.Sort(int, int, IComparer) , ArrayList.Synchronized(ArrayList) , ArrayList.Synchronized(IList) , ArrayList.ToArray() , ArrayList.ToArray(Type) , ArrayList.TrimToSize() , ArrayList.Capacity , ArrayList.Count , ArrayList.IsFixedSize , ArrayList.IsReadOnly , ArrayList.IsSynchronized , ArrayList.this[int] , ArrayList.SyncRoot , object.Equals(object) , object.Equals(object, object) , object.GetHashCode() ,

object.GetType() , object.MemberwiseClone() , object.ReferenceEquals(object, object) , object.ToString()

Constructors

ownStoredProgram(ICanvas)

Constructor. Initializes the canvas and resets the program

```
public ownStoredProgram(ICanvas canvas)
```

Parameters

`canvas` ICanvas

The canvas to be used for output

Properties

PC

```
public override int PC { get; set; }
```

Property Value

int

drawingCanvas

```
public ICanvas drawingCanvas { get; set; }
```

Property Value

ICanvas

Methods

AddVariable(Evaluation)

Adds a variable to the variable list

```
public override void AddVariable(Evaluation Variable)
```

Parameters

`Variable` Evaluation

The variable to be added

EvaluateExpression(string)

Converts an expression into a single evaluated value

```
public override string EvaluateExpression(string Exp)
```

Parameters

`Exp` string

The expressino to be evaluated

Returns

string

Evaluated expression as a string

Exceptions

StoredProgramException

Thrown when the passed expression is not an expression, or it cannot be evaluated

FindVariable(string)

Finds the index of a variable by name

```
public override int FindVariable(string varName)
```

Parameters

`varName` string

The name of the variable

Returns

int

The index of the variable

GetVarValue(string)

Used to get the value associated with a variable

```
public override string GetVarValue(string varName)
```

Parameters

`varName` string

The name of the variable we want to find the value of

Returns

string

The value of the variable

Exceptions

StoredProgramException

Thrown when the variable does not exist.

GetVariable(string)

```
public override Evaluation GetVariable(string VarName)
```

Parameters

`VarName` string

Returns

Evaluation

ResetProgram()

Resets the program to initial state

```
public override void ResetProgram()
```

Run()

Runs the execute() method of each command

```
public override void Run()
```

Exceptions

StoredProgramException

Thrown when more than 100000 are executed

UpdateVariable(string, bool)

```
public override void UpdateVariable(string varName, bool value)
```

Parameters

`varName` string

`value` bool

UpdateVariable(string, double)

Updates variables of Type Real

```
public override void UpdateVariable(string varName, double value)
```

Parameters

`varName` string

The name of the variable

`value` double

The value to be set for the variable

Exceptions

CommandException

Thrown when the value is of the Real type

UpdateVariable(string, int)

Updates a variable of type ownInt

```
public override void UpdateVariable(string varName, int value)
```

Parameters

`varName` string

The variable to be updated

`value` int

The value to be set for the variable

VariableExists(string)

Checks to see if a specific variable exists in the variable list by name

```
public override bool VariableExists(string varName)
```

Parameters

`varName` string

The name of the variable

Returns

bool

True or false depending on if the variable has been found

Class ownWhile

Namespace: ASE_Assignment

Assembly: ASE Assignment.dll

```
public class ownWhile : ownCompoundCommand, ICommand
```

Inheritance

object ← Command ← Evaluation ← Boolean ← ConditionalCommand ← ownCompoundCommand ← ownWhile

Implements

ICommand

Inherited Members

ownCompoundCommand.CoresspondingCommand , ConditionalCommand.endLineNumber , ConditionalCommand.EndLineNumber , ConditionalCommand.Condition , ConditionalCommand.LineNumber , ConditionalCommand.CondType , ConditionalCommand.ReturnLineNumber , Boolean.Restrictions() , Boolean.BoolValue , Evaluation.expression , Evaluation.evaluatedExpression , Evaluation.varName , Evaluation.value , Evaluation.CheckParameters(string[]) , Evaluation.ProcessExpression(string) , Evaluation.Expression , Evaluation.VarName , Evaluation.Value , Evaluation.Local , Command.program , Command.parameterList , Command.parameters , Command.paramsint , Command.Set(StoredProgram, string) , Command.ProcessParameters(string) , Command.ToString() , Command.Program , Command.Name , Command.ParameterList , Command.Parameters , Command.Paramsint , object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.MemberwiseClone() , object.ReferenceEquals(object, object)

Properties

Counter

counter used to check for infinite loops

```
public int Counter { get; set; }
```

Property Value

int

Methods

Compile()

method which processes parameters.

```
public override void Compile()
```

Execute()

checks to see if the condition is false, if it is then do not execute the code from the while body

```
public override void Execute()
```

Class ownWrite

Namespace: ASE Assignment

Assembly: ASE Assignment.dll

class to allow writing text on the screen

```
public class ownWrite : Evaluation, ICommand
```

Inheritance

object ← Command ← Evaluation ← ownWrite

Implements

ICommand

Inherited Members

Evaluation.expression , Evaluation.evaluatedExpression , Evaluation.varName , Evaluation.value , Evaluation.Compile() , Evaluation.CheckParameters(string[]) , Evaluation.ProcessExpression(string) , Evaluation.Expression , Evaluation.VarName , Evaluation.Value , Evaluation.Local , Command.program , Command.parameterList , Command.parameters , Command.paramsint , Command.Set(StoredProgram, string) , Command.ProcessParameters(string) , Command.ToString() , Command.Program , Command.Name , Command.ParameterList , Command.Parameters , Command.Paramsint , object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.MemberwiseClone() , object.ReferenceEquals(object, object)

Constructors

ownWrite()

constructor which assigns a canvas to the singleton draws object

```
public ownWrite()
```

Methods

Execute()

evaluates any expression, then calls the writetext method from the draws class

```
public override void Execute()
```

Namespace UnitTestsASE

Classes

ArrayTests

Test class to test all the functionalities of the Array class

CircleTests

Test Class used to test the Circle command.

ColourTests

Test class used to test the setColour methods.

DrawtoTests

Test Class used to test the DrawTo command

ForTests

Test class to test all the functionalities of the For class

IfTests

Test class to check all functionalities of the If and Else class

IntTests

Test class to check all the functionalities of the integer class

MoveToTests

Test Class used to test the MoveTo command.

MultilineTest

Test class to test the multiline functionality of the program

RealTests

Test class to test all the functionalities of the real class

RectangleTests

Test class used to test the Rect method.

ResetTest

Test class used to test the reset method.

TriangleTests

Test class to test the Triangle method

WhileTests

Test class to test all the functionalities of the While class

sqrtTest

Test class to test the functionalities of the sqrt class

Class ArrayTests

Namespace: UnitTestsASE

Assembly: UnitTestsASE.dll

Test class to test all the functionalities of the Array class

```
[TestClass]
public class ArrayTests
```

Inheritance

object ← ArrayTests

Inherited Members

object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.MemberwiseClone() , object.ReferenceEquals(object, object) , object.ToString()

Methods

CheckIfArrayWasAdded()

test method to check if an array was successfully added to the program.

```
[TestMethod]
public void CheckIfArrayWasAdded()
```

CheckIfPeekGetsCorrectValue()

test method to see if the peek command correctly receives a value from the array

```
[TestMethod]
public void CheckIfPeekGetsCorrectValue()
```

CheckIfPokeEvaluatesExpression()

test method to see if expressions can be used to access an element in the array

```
[TestMethod]
public void CheckIfPokeEvaluatesExpression()
```

CheckIfPokeSetsCorrectValue()

test method to see if the poke command correctly sets a value at a specific location in the array

```
[TestMethod]
public void CheckIfPokeSetsCorrectValue()
```

OutOfBoundsAttempt()

test method to see if attempting to access elements outside the array boundary throws an exception

```
[TestMethod]
public void OutOfBoundsAttempt()
```

Class CircleTests

Namespace: UnitTestsASE

Assembly: UnitTestsASE.dll

Test Class used to test the Circle command.

```
[TestClass]
public class CircleTests
```

Inheritance

object ← CircleTests

Inherited Members

object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.MemberwiseClone() , object.ReferenceEquals(object, object) , object.ToString()

Methods

InvalidRadiusTestThrowsCanvasException()

Test method used to see if the Circle method will throw a CanvasException when a negative radius is passed.

```
[TestMethod]
public void InvalidRadiusTestThrowsCanvasException()
```

XPositionAfterCircleCommand()

Test method used to see if the X position didn't change after calling the Circle method.

```
[TestMethod]
public void XPositionAfterCircleCommand()
```

XPositionAfterDrawToThenCircle()

Test method used to see if the X position set by the DrawTo command didn't change because of the Circle command

```
[TestMethod]
public void XPositionAfterDrawToThenCircle()
```

XPositionAfterMoveToThenCircle()

Test method used to see if the X position set by the MoveTo command didn't change because of the Circle command

```
[TestMethod]
public void XPositionAfterMoveToThenCircle()
```

YPositionAfterCircleCommand()

Test method used to see if the Y position didn't change after calling the Circle method.

```
[TestMethod]
public void YPositionAfterCircleCommand()
```

YPositionAfterDraweToThenCircle()

Test method used to see if the X position set by the DrawTo command didn't change because of the Circle command

```
[TestMethod]
public void YPositionAfterDraweToThenCircle()
```

YPositionAfterMoveToThenCircle()

Test method used to see if the Y position set by the MoveTo command didn't change because of the Circle command

```
[TestMethod]
public void YPositionAfterMoveToThenCircle()
```

Class ColourTests

Namespace: UnitTestsASE

Assembly: UnitTestsASE.dll

Test class used to test the setColour methods.

```
[TestClass]
public class ColourTests
```

Inheritance

object ← ColourTests

Inherited Members

object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.MemberwiseClone() , object.ReferenceEquals(object, object) , object.ToString()

Methods

ColorTypeColorSetTest()

Test method used to see if the alternate setColour method correctly creates a pen with a specified colour of Color type

```
[TestMethod]
public void ColorTypeColorSetTest()
```

RGBColorSetTest()

Test method used to see if the setColour method correctly creates a pen with the specified RGB colour

```
[TestMethod]
public void RGBColorSetTest()
```

RGBInvalidBlueColourSetShouldThrowCanvasException()

Test method to see if blue negative and values exceeding 255 throw a CanvasException

```
[TestMethod]
public void RGBInvalidBlueColourSetShouldThrowCanvasException()
```

RGBInvalidGreenColourSetShouldThrowCanvasException()

Test method to see if green negative and values exceeding 255 throw a CanvasException

```
[TestMethod]
public void RGBInvalidGreenColourSetShouldThrowCanvasException()
```

RGBInvalidRedColourSetShouldThrowCanvasException()

Test method to see if red negative and values exceeding 255 throw a CanvasException

```
[TestMethod]
public void RGBInvalidRedColourSetShouldThrowCanvasException()
```

Class DrawtoTests

Namespace: UnitTestsASE

Assembly: UnitTestsASE.dll

Test Class used to test the DrawTo command

```
[TestClass]
public class DrawtoTests
```

Inheritance

object ← DrawtoTests

Inherited Members

object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.MemberwiseClone() , object.ReferenceEquals(object, object) , object.ToString()

Methods

DrawToXTestCorrect()

Test method used to see if the X position of the cursor correctly changes to a specified expected value after the DrawTo command is called, with the x parameter being set to the specified expected value.

```
[TestMethod]
public void DrawToXTestCorrect()
```

DrawToXTestIncorrect()

Test method used to see if a random expected value causes the test to pass when moving the cursor using the DrawTo command

```
[TestMethod]
public void DrawToXTestIncorrect()
```

DrawToYTestCorrect()

Test method used to see if the Y position of the cursor correctly changes to a specified expected value after the DrawTo command is called, with the Y parameter being set to the specified expected value.

```
[TestMethod]
public void DrawToYTestCorrect()
```

DrawToYTestIncorrect()

Test method used to see if a random expected value causes the test to pass when moving the cursor using the DrawTo command

```
[TestMethod]
public void DrawToYTestIncorrect()
```

NegativeOutofBoundsXshouldThrowCanvasException()

Test method used to see if calling the DrawTo method with X parameter values being negative throws a CanvasException

```
[TestMethod]
public void NegativeOutofBoundsXshouldThrowCanvasException()
```

NegativeOutofBoundsYshouldThrowCanvasException()

Test method used to see if calling the DrawTo method with Y parameter values being negative throws a CanvasException

```
[TestMethod]
public void NegativeOutofBoundsYshouldThrowCanvasException()
```

PositiveOutofBoundsYshouldThrowCanvasException()

Test method used to see if calling the DrawTo method with Y parameter values exceeding the canvas boundary throws a CanvasException

```
[TestMethod]
public void PositiveOutofBoundsYshouldThrowCanvasException()
```

positiveOutofBoundsXshouldThrowCanvasException()

Test method used to see if calling the DrawTo method with X parameter values exceeding the canvas boundary throws a CanvasException

```
[TestMethod]
public void positiveOutofBoundsXshouldThrowCanvasException()
```

Class ForTests

Namespace: UnitTestsASE

Assembly: UnitTestsASE.dll

Test class to test all the functionalities of the For class

```
[TestClass]
public class ForTests
```

Inheritance

object ← ForTests

Inherited Members

object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.MemberwiseClone() , object.ReferenceEquals(object, object) , object.ToString()

Methods

TestToSeeCorrectLoopAmountCustomStep()

test method used to see if the for loop correctly operates when stepping by an increment bigger than 1

```
[TestMethod]
public void TestToSeeCorrectLoopAmountCustomStep()
```

TestToSeeCorrectLoopAmountNegativeStep()

test method used to see if the for loop correctly iterates backwards

```
[TestMethod]
public void TestToSeeCorrectLoopAmountNegativeStep()
```

TestToSeeCorrectLoopAmountNoStep()

test method used to see if the for loop correctly iterates through its expression

```
[TestMethod]
public void TestToSeeCorrectLoopAmountNoStep()
```

TestToSeeInfiniteLoop()

test method to see if the used expression will result in an infinite loop

```
[TestMethod]
public void TestToSeeInfiniteLoop()
```

Class IfTests

Namespace: UnitTestsASE

Assembly: UnitTestsASE.dll

Test class to check all functionalities of the If and Else class

```
[TestClass]
public class IfTests
```

Inheritance

object ← IfTests

Inherited Members

object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.MemberwiseClone() , object.ReferenceEquals(object, object) , object.ToString()

Methods

TestToSeeWhichCodeExecutedWhenConditionFail()

Test method to see which fragment of code will be executed when the if condition results in false

```
[TestMethod]
public void TestToSeeWhichCodeExecutedWhenConditionFail()
```

TestToSeeWhichCodeExecutedWhenConditionPass()

Test method to see which fragment of code will be executed when the if condition results in true

```
[TestMethod]
public void TestToSeeWhichCodeExecutedWhenConditionPass()
```

Class IntTests

Namespace: UnitTestsASE

Assembly: UnitTestsASE.dll

Test class to check all the functionalities of the integer class

```
[TestClass]
public class IntTests
```

Inheritance

object ← IntTests

Inherited Members

object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.MemberwiseClone() , object.ReferenceEquals(object, object) , object.ToString()

Methods

CheckIfIntAddedToVariableList()

test method which checks if a integer variable was correctly added to a list of variables

```
[TestMethod]
public void CheckIfIntAddedToVariableList()
```

CheckIfIntExpressionWasEvaluated()

test method which checks if expressions are evaluated into values

```
[TestMethod]
public void CheckIfIntExpressionWasEvaluated()
```

CheckIfIntReinitializationWillThrowException()

test method used to check if reinitalization of a variable will throw and exception

```
[TestMethod]
public void CheckIfIntReinitializationWillThrowException()
```

CheckIfIntValueDoesNotAllowRealValues()

test method used to check if passing a double value will throw an exception

```
[TestMethod]
public void CheckIfIntValueDoesNotAllowRealValues()
```

CheckIfIntValueWasChanged()

test method to see if updating the variable value will be reflected in the program.

```
[TestMethod]
public void CheckIfIntValueWasChanged()
```

CheckIfIntValueWasSet()

test method to see if a value was correctly assigned to an int variable in the declaration

```
[TestMethod]
public void CheckIfIntValueWasSet()
```

CheckIfIntValueWorksWithMoveToAndDrawTo()

test method to see if integer variables can be used with other commands like moveTo and drawTo

```
[TestMethod]
public void CheckIfIntValueWorksWithMoveToAndDrawTo()
```

Class MoveToTests

Namespace: UnitTestsASE

Assembly: UnitTestsASE.dll

Test Class used to test the MoveTo command.

```
[TestClass]
public class MoveToTests
```

Inheritance

object ← MoveToTests

Inherited Members

object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.MemberwiseClone() , object.ReferenceEquals(object, object) , object.ToString()

Methods

MoveToXTestCorrect()

Test method used to see if the X position of the cursor correctly changes to a specified expected value after the MoveTo command is called, with the x parameter being set to the specified expected value.

```
[TestMethod]
public void MoveToXTestCorrect()
```

MoveToXTestIncorrect()

Test method used to see if a random expected value causes the test to pass when moving the cursor using the MoveTo command

```
[TestMethod]
public void MoveToXTestIncorrect()
```

MoveToYTestCorrect()

Test method used to see if the Y position of the cursor correctly changes to a specified expected value after the MoveTo command is called, with the Y parameter being set to the specified expected value.

```
[TestMethod]
public void MoveToYTestCorrect()
```

MoveToYTestIncorrect()

Test method used to see if a random expected value causes the test to pass when moving the cursor using the MoveTo command

```
[TestMethod]
public void MoveToYTestIncorrect()
```

NegativeOutofBoundsXshouldThrowCanvasException()

Test method used to see if calling the MoveTo method with X parameter values being negative throws a CanvasException

```
[TestMethod]
public void NegativeOutofBoundsXshouldThrowCanvasException()
```

NegativeOutofBoundsYshouldThrowCanvasException()

Test method used to see if calling the MoveTo method with Y parameter values being negative throws a CanvasException

```
[TestMethod]
public void NegativeOutofBoundsYshouldThrowCanvasException()
```

PositiveOutofBoundsYshouldThrowCanvasException()

Test method used to see if calling the MoveTo method with Y parameter values exceeding the canvas boundary throws a CanvasException

```
[TestMethod]
public void PositiveOutofBoundsYshouldThrowCanvasException()
```

positiveOutofBoundsXshouldThrowCanvasException()

Test method used to see if calling the MoveTo method with X parameter values exceeding the canvas boundary throws a CanvasException

```
[TestMethod]
public void positiveOutofBoundsXshouldThrowCanvasException()
```

Class MultilineTest

Namespace: UnitTestsASE

Assembly: UnitTestsASE.dll

Test class to test the multiline functionality of the program

```
[TestClass]
public class MultilineTest
```

Inheritance

object ← MultilineTest

Inherited Members

object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.MemberwiseClone() , object.ReferenceEquals(object, object) , object.ToString()

Methods

XMultilineCheckUsingCircleMoveToRectTriangle()

Test method to see if the X position set by the MoveTo command isn't changed after calling the rectangle and triangle methods.

```
[TestMethod]
public void XMultilineCheckUsingCircleMoveToRectTriangle()
```

XMultilineCheckUsingCircleRectTriangle()

Test method to see if the X position remains the same after a circle,rectangle and triangle method calls.

```
[TestMethod]
public void XMultilineCheckUsingCircleRectTriangle()
```

XMultilineCheckUsingDrawTo()

Test class to see if the correct X value is set after multiple DrawTo method calls.

```
[TestMethod]
public void XMultilineCheckUsingDrawTo()
```

XMultilineCheckUsingDrawtoCircleRectTriangle()

Test method to see if the X position set by the DrawTo command isn't changed after calling the circle, rectangle and triangle methods.

```
[TestMethod]
public void XMultilineCheckUsingDrawtoCircleRectTriangle()
```

XMultilineCheckUsingMoveTo()

Test class to see if the correct X value is set after multiple MoveTo method calls.

```
[TestMethod]
public void XMultilineCheckUsingMoveTo()
```

XMultilineCheckUsingMoveToCircleRectTriangle()

Test method to see if the X position set by the MoveTo command isn't changed after calling the circle, rectangle and triangle methods.

```
[TestMethod]
public void XMultilineCheckUsingMoveToCircleRectTriangle()
```

YMultilineCheckUsingCircleMoveToRectTriangle()

Test method to see if the Y position set by the MoveTo command isn't changed after calling the rectangle and triangle methods.

```
[TestMethod]
```

```
public void YMultilineCheckUsingCircleMoveToRectTriangle()
```

YMultilineCheckUsingCircleRectTriangle()

Test method to see if the Y position remains the same after a circle,rectangle and triangle method calls.

```
[TestMethod]
public void YMultilineCheckUsingCircleRectTriangle()
```

YMultilineCheckUsingDrawTo()

Test class to see if the correct Y value is set after multiple DrawTo method calls.

```
[TestMethod]
public void YMultilineCheckUsingDrawTo()
```

YMultilineCheckUsingDrawtoCircleRectTriangle()

Test method to see if the X position set by the DrawTo command isn't changed after calling the circle, rectangle and triangle methods.

```
[TestMethod]
public void YMultilineCheckUsingDrawtoCircleRectTriangle()
```

YMultilineCheckUsingMoveTo()

Test class to see if the correct Y value is set after multiple MoveTo method calls.

```
[TestMethod]
public void YMultilineCheckUsingMoveTo()
```

YMultilineCheckUsingMoveToCircleRectTriangle()

Test method to see if the Y position set by the MoveTo command isn't changed after calling the circle, rectangle and triangle methods.

```
[TestMethod]
public void YMultilineCheckUsingMoveToCircleRectTriangle()
```

Class RealTests

Namespace: UnitTestsASE

Assembly: UnitTestsASE.dll

Test class to test all the functionalities of the real class

```
[TestClass]
public class RealTests
```

Inheritance

object ← RealTests

Inherited Members

object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.MemberwiseClone() , object.ReferenceEquals(object, object) , object.ToString()

Methods

CheckIfRealAddedToVariableList()

test method which checks if a real variable was correctly added to a list of variables

```
[TestMethod]
public void CheckIfRealAddedToVariableList()
```

CheckIfRealExpressionWasEvaluated()

test method which checks if expressions are evaluated into values

```
[TestMethod]
public void CheckIfRealExpressionWasEvaluated()
```

CheckIfRealReinitializationWillThrowException()

test method used to check if reinitalization of a variable will throw and exception

```
[TestMethod]
public void CheckIfRealReinitializationWillThrowException()
```

CheckIfRealValueDoesAllowIntValues()

test method used to check if passing a integer value will convert it into a double

```
[TestMethod]
public void CheckIfRealValueDoesAllowIntValues()
```

CheckIfRealValueWasChanged()

test method to see if updating the variable value will be reflected in the program.

```
[TestMethod]
public void CheckIfRealValueWasChanged()
```

CheckIfRealValueWasSet()

test method to see if a value was correctly assigned to a real variable in the declaration

```
[TestMethod]
public void CheckIfRealValueWasSet()
```

Class RectangleTests

Namespace: UnitTestsASE

Assembly: UnitTestsASE.dll

Test class used to test the Rect method.

```
[TestClass]
public class RectangleTests
```

Inheritance

object ← RectangleTests

Inherited Members

object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.MemberwiseClone() , object.ReferenceEquals(object, object) , object.ToString()

Methods

RectangleThrowsExceptionWhenNegativeHeightIsPassed()

Test method used to see if the Rect method throws a CanvasException when a negative Height is passed

```
[TestMethod]
public void RectangleThrowsExceptionWhenNegativeHeightIsPassed()
```

RectangleThrowsExceptionWhenNegativeWidthIsPassed()

Test method used to see if the Rect method throws a CanvasException when a negative Width is passed

```
[TestMethod]
public void RectangleThrowsExceptionWhenNegativeWidthIsPassed()
```

XPositionAfterDrawToThenRectangleCommand()

Test method used to see if the X position set by the DrawTo command didn't change because of the Rect command

```
[TestMethod]
public void XPositionAfterDrawToThenRectangleCommand()
```

XPositionAfterMoveToThenRectangleCommand()

Test method used to see if the X position set by the MoveTo command didn't change because of the Rect command

```
[TestMethod]
public void XPositionAfterMoveToThenRectangleCommand()
```

XPositionAfterRectangleCommand()

Test method used to see if the X position didn't change after calling the Rect method.

```
[TestMethod]
public void XPositionAfterRectangleCommand()
```

YPositionAfterDrawToThenRectangleCommand()

Test method used to see if the Y position set by the DrawTo command didn't change because of the Rect command

```
[TestMethod]
public void YPositionAfterDrawToThenRectangleCommand()
```

YPositionAfterMoveToThenRectangleCommand()

Test method used to see if the Y position set by the MoveTo command didn't change because of the Rect command

```
[TestMethod]
public void YPositionAfterMoveToThenRectangleCommand()
```

YPositionAfterRectangleCommand()

Test method used to see if the Y position didn't change after calling the Rect method.

```
[TestMethod]
public void YPositionAfterRectangleCommand()
```

Class ResetTest

Namespace: UnitTestsASE

Assembly: UnitTestsASE.dll

Test class used to test the reset method.

```
[TestClass]
public class ResetTest
```

Inheritance

object ← ResetTest

Inherited Members

object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.MemberwiseClone() , object.ReferenceEquals(object, object) , object.ToString()

Methods

XResetTestAfterMoving()

Test method used to see if the X position correctly changed from the reset command after a movement command was called

```
[TestMethod]
public void XResetTestAfterMoving()
```

XResetTestBeforeMoving()

Test method used to see if the X position didn't change from the reset command before any movement command was called

```
[TestMethod]
public void XResetTestBeforeMoving()
```

YResetTestAfterMoving()

Test method used to see if the Y position correctly changed from the reset command after a movement command was called

```
[TestMethod]
public void YResetTestAfterMoving()
```

YResetTestBeforeMoving()

Test method used to see if the Y position didn't change from the reset command before any movement command was called

```
[TestMethod]
public void YResetTestBeforeMoving()
```

Class TriangleTests

Namespace: UnitTestsASE

Assembly: UnitTestsASE.dll

Test class to test the Triangle method

```
[TestClass]
public class TriangleTests
```

Inheritance

object ← TriangleTests

Inherited Members

object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.MemberwiseClone() , object.ReferenceEquals(object, object) , object.ToString()

Methods

TriangleThrowsExceptionWhenNegativeHeightIsPassed()

Test method to see if the Tri method throws a CanvasException when negative height is passed

```
[TestMethod]
public void TriangleThrowsExceptionWhenNegativeHeightIsPassed()
```

TriangleThrowsExceptionWhenNegativeWidthIsPassed()

Test method to see if the Tri method throws a CanvasException when negative width is passed

```
[TestMethod]
public void TriangleThrowsExceptionWhenNegativeWidthIsPassed()
```

XPositionAfterDrawToThenTriangleCommand()

Test method used to see if the X position set by the DrawTo command didn't change because of the Tri command

```
[TestMethod]
public void XPositionAfterDrawToThenTriangleCommand()
```

XPositionAfterMoveToThenTriangleCommand()

Test method used to see if the X position set by the MoveTo command didn't change because of the Tri command

```
[TestMethod]
public void XPositionAfterMoveToThenTriangleCommand()
```

XPositionAfterTriangleCommand()

Test method to see if the X position didn't change after calling the Triangle method.

```
[TestMethod]
public void XPositionAfterTriangleCommand()
```

YPositionAfterDrawToThenTriangleCommand()

Test method used to see if the Y position set by the DrawTo command didn't change because of the Tri command

```
[TestMethod]
public void YPositionAfterDrawToThenTriangleCommand()
```

YPositionAfterMoveToThenTriangleCommand()

Test method used to see if the Y position set by the MoveTo command didn't change because of the Tri command

```
[TestMethod]
public void YPositionAfterMoveToThenTriangleCommand()
```

YPositionAfterTriangleCommand()

Test method to see if the Y position didn't change after calling the Triangle method.

```
[TestMethod]
public void YPositionAfterTriangleCommand()
```

Class WhileTests

Namespace: UnitTestsASE

Assembly: UnitTestsASE.dll

Test class to test all the functionalities of the While class

```
[TestClass]
public class WhileTests
```

Inheritance

object ← WhileTests

Inherited Members

object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.MemberwiseClone() , object.ReferenceEquals(object, object) , object.ToString()

Methods

TestToCheckForInfiniteLoop()

Test method to see if the program detects an infinite loop

```
[TestMethod]
public void TestToCheckForInfiniteLoop()
```

TestToSeeHowManyTimesTheLoopWillGoOnFor()

test to see if the while will execute the correct code when its condition is true

```
[TestMethod]
public void TestToSeeHowManyTimesTheLoopWillGoOnFor()
```

TestToSeeHowManyTimesTheLoopWillGoOnFornegative()

test to see if negative values can be used in the condition

```
[TestMethod]
public void TestToSeeHowManyTimesTheLoopWillGoOnFornegative()
```

TestToSeeIfLoopWillBeSkipped()

test to see if the while will be executed if the condition is not true

```
[TestMethod]
public void TestToSeeIfLoopWillBeSkipped()
```

Class sqrtTest

Namespace: UnitTestsASE

Assembly: UnitTestsASE.dll

Test class to test the functionalities of the sqrt class

```
[TestClass]
public class sqrtTest
```

Inheritance

object ← sqrtTest

Inherited Members

object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.MemberwiseClone() , object.ReferenceEquals(object, object) , object.ToString()

Methods

TestSqrt()

test method used to check if the sqrt class correctly square roots a value

```
[TestMethod]
public void TestSqrt()
```

TestSqrtNegative()

test method used to check if the sqrt class correctly throws an exception when a negative value is attempting to be processed

```
[TestMethod]
public void TestSqrtNegative()
```